

Bad Smells in Code

❖ Duplicated Code

- ❖ bad because if you modify one instance of duplicated code but not the others, you (may) have introduced a bug!

❖ Long Method

- ❖ long methods are more difficult to understand
 - ❖ performance concerns with respect to lots of short methods are largely obsolete

Bad Smells in Code

❖ Large Class

- ❖ Large classes try to do too much, which reduces cohesion

❖ Long Parameter List

- ❖ hard to understand, can become inconsistent

❖ Divergent Change

- ❖ Related to cohesion
- ❖ symptom: one type of change requires changing one subset of methods; another type of change requires changing another subset

Bad Smells in Code

Shotgun Surgery

- a change requires lots of little changes in a lot of different classes

Feature Envy

- A method requires lots of information from some other class
 - move it closer!

Data Clumps

- attributes that clump together (are used together) but are not part of the same class

Bad Smells in Code

Primitive Obsession

- characterized by a reluctance to use classes instead of primitive data types

Switch Statements

- Switch statements are often duplicated in code; they can typically be replaced by use of polymorphism (let OO do your selection for you!)

Parallel Inheritance Hierarchies

- Similar to Shotgun Surgery; each time I add a subclass to one hierarchy, I need to do it for all related hierarchies
 - Note: some design patterns encourage the creation of parallel inheritance hierarchies (so they are not always bad!)

Bad Smells in Code

❦ Lazy Class

- ❦ A class that no longer “pays its way”

- ❦ e.g. may be a class that was downsized by a previous refactoring, or represented planned functionality that did not pan out

❦ Speculative Generality

- ❦ “Oh I think we need the ability to do this kind of thing someday”

❦ Temporary Field

- ❦ An attribute of an object is only set in certain circumstances; but an object should need all of its attributes

Bad Smells in Code

❦ Message Chains

- ❦ a client asks an object for another object and then asks that object for another object etc. Bad because client depends on the structure of the navigation

❦ Middle Man

- ❦ If a class is delegating more than half of its responsibilities to another class, do you really need it? (involves trade-offs, some design patterns encourage this (e.g. Decorator))

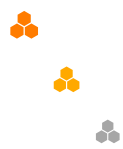
❦ Inappropriate Intimacy

- ❦ Pairs of classes that know too much about each other's private details (loss of encapsulation; change one class, the other has to change)

Bad Smells in Code

- Alternative Classes with Different Interfaces
 - Symptom: Two or more methods do the same thing but have different signatures for what they do
- Incomplete Library Class
 - A framework class doesn't do everything you need

Bad Smells in Code

- Data Class
 - These are classes that have fields, getting and setting methods for the fields, and nothing else; they are data holders, but objects should be about data AND behavior
 - Refused Bequest
 - A subclass ignores most of the functionality provided by its superclass
 - Subclass may not pass the "IS-A" test
- 

The Catalog

- The refactoring book has 72 refactoring patterns!
- I'm only going to cover a few of the more common ones, including
 - Extract Method
 - Replace Temp with Query
 - Move Method
 - Replace Conditional with Polymorphism
 - Introduce Null Object



Extract Method

- You have a code fragment that can be grouped together
- Turn the fragment into a method whose name explains the purpose of the fragment
- Example, next slide



Extract Method, continued

```
void printOwing(double amount) {  
    printBanner()  
    //print details  
    System.out.println("name: " + _name);  
    System.out.println("amount: " + amount);  
}  
=====
```

```
void printOwing(double amount) {  
    printBanner()  
    printDetails(amount)  
}  
  
void printDetails(double amount) {  
    System.out.println("name: " + _name);  
    System.out.println("amount: " + amount);  
}
```

Replace Temp with Query

- ❖ You are using a temporary variable to hold the result of an expression
- ❖ Extract the expression into a method; Replace all references to the temp with the expression. The new method can then be used in other methods
- ❖ Example, next slide

Replace Temp with Query, continued

```
double basePrice = _quantity * _itemPrice
if (basePrice > 1000)
    return basePrice * 0.95;
else
    return basePrice * 0.98;
=====
if (basePrice() > 1000)
    return basePrice() * 0.95;
else
    return basePrice() * 0.98;
...
double basePrice() {
    return _quantity * _itemPrice;
}
```



Move Method

- ⬢ A method is using more features (attributes and operations) of another class than the class on which it is defined
- ⬢ Create a new method with a similar body in the class it uses most. Either turn the old method into a simple delegation, or remove it altogether



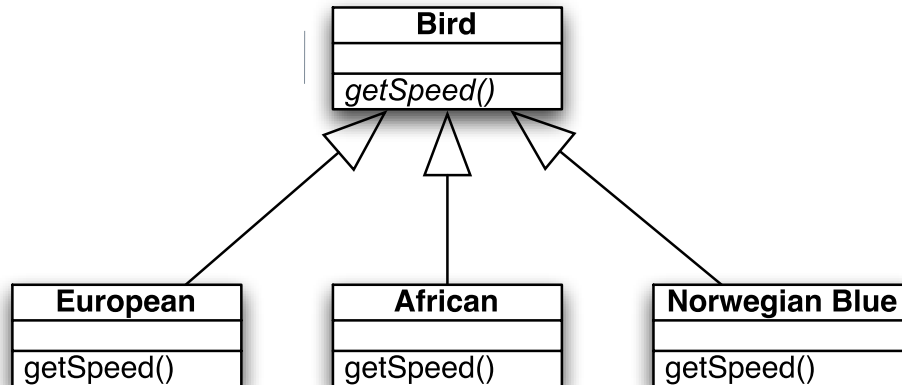
Replace Conditional with Polymorphism

- You have a conditional that chooses different behavior depending on the type of an object
- Move each “leg” of the conditional to an overriding method in a subclass. Make the original method abstract

Replace Conditional with Polymorphism, continued

```
double getSpeed() {
    switch (_type) {
        case EUROPEAN:
            return getBaseSpeed();
        case AFRICAN:
            return getBaseSpeed() - getLoadFactor() *
                _numberOfCoconuts;
        case NORWEGIAN_BLUE:
            return (_isNailed) ? 0 : getBaseSpeed(_voltage);
    }
    throw new RuntimeException("Unreachable")
}
```


Replace Conditional with Polymorphism, continued

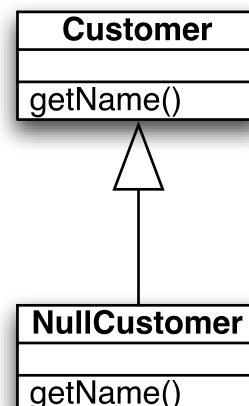


Introduce Null Object

- Repeated checks for a null value (see below)
- Rather than returning a null value from `findCustomer()` return an instance of a "null customer" object

```

...
Customer c = findCustomer(...);
...
if (customer == null) {
    name = "occupant"
} else {
    name = customer.getName()
}
if (customer == null) {
    ...
  
```



Introduce Null Object

```
public class nullCustomer {  
    public String getName() { return "occupant";}  
}  
=====
```

`Customer c = findCustomer(...);`
`name = c.getName();`

- 🍷 The conditional goes away entirely!!
- 🍷 In Fowler's book, this technique is presented as a refactoring; in other contexts, its presented as a design pattern
 - 🍷 Either way, its very useful!

